UAVSAR Data Preprocessing for Polarimetric Despeckling (PolNBDNet)

---

1. Get the UAVSAR SLC data

What: UAVSAR is NASA/JPL's airborne, fully polarimetric L-band (1.26 GHz) SAR. The single-look complex (SLC) product gives you one complex value per pixel per polarization (HH, HV, VH, VV), at full resolution, before any multilooking or symmetrization has been applied.

Why: SLC is the rawest usable polarimetric product. Because nothing has been averaged yet, you get to choose your own multilook window, and — critically for training data — you can split the synthetic aperture into independent sub-looks of the same scene (Step 11). Neither is possible once you start from the already-multilooked MLC product.

How: 1. Search the UAVSAR data portal (JPL) or ASF Vertex by flight line / scene name. 2. For each scene, download: - the `.ann` file (always — read this first), - the four SLC files: one per polarization, `HH`, `HV`, `VH`, `VV` (each complex), - `.inc`, `.slope`, `.hgt` if you plan terrain/incidence-angle checks. 3. Organize files in a fixed structure so every later step can find them predictably:

```
data/
  raw/<scene_id>/          # untouched downloads, read-only
  interim/<scene_id>/      # masked, multilooked, cropped
  processed/<scene_id>/    # model-ready tiles + metadata json
  splits/                  # train/val/test scene lists
```

4. Record scene ID, acquisition date, heading, and pixel spacing in a `metadata.json` per scene. You'll need this later to avoid mixing scenes across train/val/test.

---

2. Read the SLC binary data correctly

What: Loading the four raw SLC polarization files into complex arrays with the correct shape and byte order.

Why: UAVSAR SLC files are raw little-endian `complex64` binary with no embedded header. Getting the shape or dtype wrong silently produces garbage that can still look like an image if you're not careful.

How: 1. Open the `.ann` file and find the SLC row/column counts (entries such as `slc_1_1x1 rows` / `slc_1_1x1 columns` — exact key names can vary slightly by data version; UAVSAR SLC scenes are also sometimes split into multiple range segments, so check whether your scene has more than one). 2. Read each polarization file as complex little-endian:

```python
import numpy as np

def read_uavsar_slc(path, rows, cols):
    return np.fromfile(path, dtype="<c8").reshape(rows, cols)
```

```
hh = read_uavsar_slc(f"{stem}_HH_{tag}.slc", rows, cols)
hv = read_uavsar_slc(f"{stem}_HV_{tag}.slc", rows, cols)
vh = read_uavsar_slc(f"{stem}_VH_{tag}.slc", rows, cols)
vv = read_uavsar_slc(f"{stem}_VV_{tag}.slc", rows, cols)
```

3. Immediately sanity-check: shapes match the `.ann`; `|hh|**2` as a quick dB image (`10*np.log10(np.abs(hh)**2)`) looks like a SAR scene, not noise; `hv` and `vh` should be close but not identical (SLC is not yet symmetrized — that's expected and is handled in Step 4).
4. For large scenes, use `np.memmap` or process in blocks instead of loading everything into RAM at once.

---

3. Mask invalid pixels

What: A boolean array marking which pixels contain real, usable data.

Why: Every scene has edges, no-data borders, and sometimes dropped pixels. If these are not excluded, they corrupt every later statistic — normalization ranges, filter outputs, and training-tile validity all get skewed by garbage values.

How: 1. Mask any pixel where a channel is NaN, inf, exactly zero, or equal to the product's documented no-data fill value (check the `.ann`). 2. Mask the scene edges and near/far-range borders where the data are only partially populated. 3. Optionally erode the mask by a few pixels so downstream filters don't bleed in edge artefacts. 4. Store the mask; every later step (multilooking, filtering, statistics, tiling) must respect it.

---

4. Form the multilooked covariance matrix

What: Building the 3x3 polarimetric covariance matrix `C` at each output pixel by spatially averaging outer products of the SLC scattering vector over a chosen window — this is the multilooking step that MLC data already has done for you, but which you must do yourself starting from SLC.

Why: A single SLC pixel gives one complex scattering vector, not a statistically meaningful covariance matrix — averaging over multiple looks is what makes the covariance estimate usable at all. This is also where you symmetrize HV and VH (SAR reciprocity: they should be statistically equal after averaging, so they're merged into one cross-term) and where you get to choose your look count, which MLC data fixes for you.

How: 1. Build the per-pixel scattering vector using the lexicographic basis `k = [HH, sqrt(2)*HV_sym, VV]`, where `HV_sym = (HV + VH) / 2`. 2. Form the outer product `k k^H` at every pixel (a 3x3 complex matrix per pixel). 3. Spatially average `k k^H` over a chosen window (e.g. 5x5 or 7x7, matching your desired number of looks `L`) to get `C = mean(k k^H)` over that window. Average the complex matrix elements directly — never average derived dB or coherence values instead. 4. This gives you one multilooked covariance matrix `C` per output pixel, at reduced resolution (or same resolution with overlapping windows, if you prefer). 5. Check: diagonal elements of `C` are real and non-negative; `C` is Hermitian; a Pauli RGB of `C` looks like a recognizable, correctly speckled SAR scene.

---

5. Handle radiometry correctly

What: Keeping the data in the right units and accounting for incidence-angle effects before any further averaging or filtering.

Why: UAVSAR power is linear, not dB. Averaging or filtering in dB (a common mistake) biases the result, because `mean(dB)` is not the same as `10*log10(mean(linear))`. Separately, backscatter naturally varies with incidence angle (roughly 25°-65° across the swath); if your task is sensitive to that (e.g. classifying across the full swath), an uncorrected range trend will look like a real signal to the network when it isn't.

How: 1. Do all averaging, filtering, and matrix math in **linear power**. Convert to dB only for visualization or as a final, one-time input transform. 2. Use the `.inc` file to check whether backscatter trends with incidence angle across your scenes. If it matters for your task, normalize consistently or add incidence angle as an extra input — pick one approach and use it everywhere. 3. In steep terrain, use `.slope` and `.hgt` to flag layover/shadow pixels rather than trusting them as normal terrain.

---

6. Decide how to handle speckle — and why polarimetric speckle is different

What: Speckle is the grainy multiplicative noise inherent to coherent SAR imaging. For a **single-channel** despeckling model, training pairs are usually made by multiplying a clean intensity by a random Gamma-distributed number. For **polarimetric** data (your case), this shortcut is wrong, because it treats the six covariance elements as independent numbers instead of one coherent matrix per pixel.

Why this distinction matters: the off-diagonal covariance terms carry the **phase relationship** between polarization channels — this is exactly what polarimetric decompositions (Pauli, Freeman-Durden, H/A/alpha) depend on. If you add noise to each of the six channels separately: - the result is very likely no longer a valid covariance matrix (not Hermitian positive semi-definite), - the cross-channel phase is destroyed or randomized, and the network learns nothing meaningful about scattering mechanisms.

How to check whether a pipeline (yours or someone else's) is doing this correctly: - Is noise applied to the full 3x3 matrix jointly, or to each channel in a loop with independent random draws? A loop over channels is the wrong approach. - After adding noise, is the result still a valid covariance matrix (Hermitian, positive semi-definite, |coherence| ≤ 1)? If not, something is broken.

Because you have SLC data, you actually have two correct options: **simulate** speckle synthetically from a heavily multilooked reference (Step 8), or use **real independent looks** taken directly from the SLC data (Step 11). The second option is usually preferable when SLC is available, since it needs no synthetic noise model at all — see Step 11.

---

7. Choose the model's input representation

What: Converting the 3x3 covariance matrix `C` into a fixed 9-channel numeric representation the network can consume.

Why: The network needs one consistent numeric layout for every pixel — raw linear power spans a huge dynamic range, and the six unique elements of `C` are not directly comparable in scale (three real

powers, three complex correlations). Separating “how much power” from “how correlated the channels are” makes both easier for a network to learn and easier to constrain to valid values later.

How: 1. Use `cov_to_repr()` from `pol_nbdnet.py` to convert `C` into 3 normalized-dB diagonal powers + 6 real/imaginary parts of the complex coherences (`g_ij = C_ij / sqrt(C_ii * C_jj)`, each bounded in magnitude by 1). `repr_to_cov()` converts back. 2. Fix the dB normalization range (`DB_MIN`, `DB_MAX` in the script) from your actual data statistics, computed on valid pixels only, and reuse the same constants everywhere — training, validation, and inference.

---

8. Build the “clean” reference

What: A stand-in for the true, speckle-free covariance matrix at each pixel — call it `C_ref`. This becomes the training target if you’re using the synthetic-noise route (Step 9). If you use the real-independent-looks route instead (Step 11), you can skip this step.

Why: A genuinely speckle-free SAR image doesn’t exist in nature; speckle is fundamental to coherent imaging, not removable sensor noise. Since you can’t obtain real ground truth, you approximate it by heavily averaging a real scene until residual speckle is small.

How: 1. Pick spatially **homogeneous** patches — large fields, stable forest, calm water. Avoid edges and texture-rich areas; averaging across a real edge teaches the network to blur edges. 2. From your SLC data, form `C_ref` using a much larger multilook window than Step 4’s target window (e.g. 15x15 or more instead of 5x5), following the same outer-product-averaging procedure. 3. Discard `C_ref` patches with too many masked pixels. 4. Check visually: a Pauli RGB of `C_ref` should look like a smoothed version of the original, same features, not blocky or over-erased.

---

9. Generate synthetic noisy data from the clean reference (Wishart simulation)

What: Starting from `C_ref`, generate `C_noisy` with statistically correct multilook polarimetric speckle — the polarimetric equivalent of the Gamma-multiplier trick, done jointly across channels instead of per-channel.

Why: Real polarimetric SAR speckle at the covariance level follows a complex Wishart distribution. Simulating it correctly means drawing correlated complex scattering vectors from `C_ref` and averaging their outer products — this preserves the cross-channel phase relationships that per-channel noise would destroy (Step 6).

How: 1. Compute the Cholesky factor of `C_ref`: `C_ref = L L^H` (add small diagonal jitter first if `C_ref` is near-singular). 2. For each of `L` looks, draw `n ~ CN(0, I)` (three independent complex standard normal values) and set `z = L @ n`, so `z` has covariance `C_ref`. 3. Average the outer products over `L` looks: `C_noisy = (1/L) * sum(z_l @ z_l^H)`. 4. Repeat per pixel (or per patch) to build a full noisy scene. Vary `L` across samples (e.g. randomly between 1 and 8) if you want the network to generalize across look counts, rather than fixing one value. 5. Use `simulate_polsar_speckle()` in `pol_nbdnet.py` — it implements exactly this. 6. Check with `check_validity()` (also in the script): `C_noisy` should be Hermitian with no negative eigenvalues beyond numerical tolerance. Many violations mean noise was added incorrectly (per-channel instead of jointly).

Skip this step entirely if you're using the real-independent-looks route (Step 11) instead.

---

10. Tile into matched training pairs

What: Cutting the full noisy and clean (or the two independent-look) scenes into small, fixed-size patches for training.

Why: Full scenes don't fit in memory/GPU per batch, and CNNs train more efficiently on many smaller patches than on a few huge images.

How: 1. Match your tile size to the network's `Input(shape=(...))` — `(64, 64, 9)` for the model in `pol_nbdnet.py`. 2. Cut the **same pixel region** from both members of the pair — they must stay pixel-aligned. Slide with overlap (e.g. every 32 pixels for a 64x64 window) if you want more samples. 3. Convert both tiles to the 9-channel representation and discard tiles where valid-pixel fraction falls below a threshold (e.g. 90%). 4. Save pairs with scene ID and tile origin recorded, so predictions can later be traced back and stitched. 5. Spot-check a few pairs as Pauli RGB, side by side — same content, nothing misaligned.

---

11. Split into train / validation / test

What: Dividing tiles into non-overlapping sets.

Why: Splitting by random tile lets neighboring tiles from the same scene land in both train and test. Since they share real texture and partially even share speckle correlation, test performance would look better than it really is — this is data leakage.

How: 1. Split **by scene**, not by tile — assign whole flight lines/scenes to train, validation, or test before tiling. 2. Keep all passes of the same flight line in one split if a line was imaged multiple times. 3. Confirm no scene ID appears in more than one split before training starts.

---

12. Recommended for SLC data: real independent looks (Noise2Noise), no synthetic noise

What: Train directly on two **independently noisy** real observations of the same scene, taken from your SLC data — no `C_ref` approximation and no synthetic Wishart draw needed at all.

Why this is the better fit now that you have SLC: approximating "clean" data (Step 8) is always a compromise, and synthetic speckle (Step 9), while statistically correct, is still simulated. Since you already have SLC, you can instead split the real synthetic aperture into two independent halves and get two genuinely independent noisy observations of the same real scene — more realistic, and it needs no noise model at all.

How: 1. Split the synthetic aperture into two independent sub-apertures (e.g. first half / second half of the azimuth spectrum, via an azimuth FFT, filter, and inverse FFT on the SLC data) — same scene, statistically independent speckle. 2. Form covariance matrices from each sub-aperture using the same multilooking procedure as Step 4: `C_a` and `C_b`. 3. Train with `C_a` as input and `C_b` as target (or both directions), using the Wishart/KL loss (`wishart_distance()` in

`pol_nbdnet.py`) between prediction and `C_b`. 4. Skip Steps 8 and 9 entirely; go straight to tiling (Step 10) and splitting (Step 11) using `C_a`/`C_b` pairs.

Trade-off: sub-aperture splitting reduces azimuth resolution, and the FFT/filter/inverse-FFT step is more involved than the synthetic Wishart route (Steps 8-9). If your students haven't covered azimuth-spectrum processing yet, start with the synthetic route and treat this as the next step up.

---

13. Final checklist before training

Run all of these on a sample batch before starting training:

- ☐ Array shapes match the `.ann`; dtypes and byte order are correct (Step 2).
- ☐ Diagonal power channels are non-negative; no NaN/inf inside the valid mask.
- ☐ `C` from Step 4 is Hermitian, with HV/VH correctly symmetrized.
- ☐ If using synthetic noise: `C_noisy` and `C_ref` are both Hermitian and positive semi-definite (`check_validity()` reports no violations), and a Pauli RGB of `C_noisy` looks like a speckled version of the same scene as `C_ref`, not random colored noise.
- ☐ If using real independent looks: `C_a` and `C_b` show the same scene content with visibly independent speckle patterns.
- ☐ Tiles in each pair are pixel-aligned, from the same scene location.
- ☐ No scene ID appears in more than one of train/val/test.
- ☐ `stats.json` (normalization constants) and `metadata.json` (scene info) are saved alongside the processed tiles, so results are reproducible.

Once every box is checked, the data is ready to train PolNBDNet (model and losses in `pol_nbdnet.py`).

14. Common mistakes to avoid

1. Treating power as dB, or averaging in dB.
2. Wrong byte order or dtype when reading raw SLC files.
3. Forgetting to symmetrize HV and VH when building the covariance matrix.
4. Adding speckle to each covariance channel independently instead of jointly (Wishart draw).
5. Building `C_ref` from a patch that includes a real edge or texture, blurring it into the "clean" target.
6. Computing normalization statistics on validation/test data instead of training data only.
7. Random tile-level train/test splits that leak across scenes.
8. Mixing scenes processed with different multilook windows in one dataset.

15. References

- Fore et al., *UAVSAR Polarimetric Calibration*, IEEE TGRS.
- UAVSAR product and data-format documentation, JPL UAVSAR site.
- Lee & Pottier, *Polarimetric Radar Imaging: From Basics to Applications*.
- Lehtinen & Munkberg et al., *Noise2Noise: Learning Image Restoration without Clean Data* (for Step 12).